Tourism Web App – Technical Specification

This specification with Node.js, PostgreSQL, Redis, server-rendered HTML and Tailwind CSS, with every file kept under 200 lines (hard limit 350).

Stack and engineering rules

One Node.js application renders HTML on the server, styles it with Tailwind, and stores data in PostgreSQL with Redis for sessions, cache and queues. No front-end framework is needed; small vanilla JavaScript modules add interactivity.

Concern	Choice	Notes
Runtime	Node.js 22 LTS, ES modules	Plain JavaScript with JSDoc types (TypeScript optional)
Web framework	Express 5	Mature, large ecosystem
Templates	Nunjucks (HTML)	Layouts, partials and macros keep views small
Styling	Tailwind CSS 4 via CLI	Built to one minified CSS file
Browser JS	Vanilla ES modules, plus htmx for partial updates	Optional; each module under 150 lines
Database	PostgreSQL 18 with node-postgres (pg)	Parameterised SQL in repository files
Migrations	node-pg-migrate	One migration file per change
Redis client	ioredis	Sessions, cache, rate limits, queues
Queue	BullMQ	Email, image processing, cache warming
Validation	Zod	One schema file per resource
Auth	express-session with connect-redis, argon2	Server-side sessions, not JWTs
Security	helmet, csrf-sync, express-rate-limit with rate-limit-redis, hpp	See Security architecture
Images	multer and sharp	Resize to WebP/AVIF, strip metadata
Email	nodemailer (SMTP or Resend)	Sent from a queue
i18n	i18next	English and Swahili JSON files
Logging	pino and pino-http	Structured JSON logs

Concern	Choice	Notes
Testing	Vitest, Supertest	Unit, integration and permission tests
Quality	ESLint, Prettier, Husky	Enforces the file-size rule

Modularity rules (enforced, not advisory)

1. One exported function per file, named after the file (`createBooking.js` exports `createBooking`).
2. Soft limit 200 lines per file; hard limit 350. ESLint `max-lines` set to warn at 200 and fail at 350, run in CI and in a pre-commit hook.
3. Max 40 lines per function, max 4 parameters (use an options object beyond that).
4. Layers only call downward: route, then controller, then service, then repository. No SQL outside repositories; no `req` or `res` outside controllers and middleware.
5. Each module (tours, bookings, blog and so on) is a self-contained folder with the same internal shape.
6. Shared code lives in `src/shared`; nothing imports from another module's internals, only from its `index.js`.
7. Long lists (menus, translations, constants) are split into data files, not inlined into logic.

Architecture

Cloudflare serves cached pages; everything else reaches Nginx, then the Node app, which talks to PostgreSQL and Redis. A separate worker process runs queue jobs.

```
Visitor / Staff browser
        |
   Cloudflare (CDN, WAF, SSL, HTML + image cache)
        |
   Nginx on the VPS (TLS, gzip/brotli, static files, rate limits)
        |
   Node.js web process (Express)        Node.js worker process (BullMQ)
        |        |                              |
   PostgreSQL   Redis  <------------------------+
   (data)       (sessions, cache, queues, rate limits)
        |
   Object storage (images, backups)
```

Layers (each calls only the one below)

Layer	Folder	Responsibility
Routes	`routes/`	Map URL and method to middleware chain and one controller
Controllers	`controllers/`	Read the request, call one service, send HTML or JSON
Services	`services/`	Business rules, transactions, cache invalidation, events
Repositories	`repositories/`	The only place with SQL; return plain objects

Layer	Folder	Responsibility
Validators	`validators/`	Zod schemas for body, query and params
Views	`views/`	Nunjucks templates, no business logic
Jobs	`jobs/`	Background work (email, images, sitemap)

Request lifecycle

1. Cloudflare returns a cached copy for public GET pages; otherwise the request goes on to Nginx.
2. Express runs global middleware: request ID, logging, helmet, compression, cookies, session, language detection.
3. The route applies its own middleware in this order: rate limit, authentication, role check, CSRF, validation.
4. The controller calls one service function.
5. The service reads Redis first for cacheable data, then calls repositories (inside a transaction for writes).
6. On write, the service clears the affected Redis keys and queues side effects (email, image work, Cloudflare cache purge).
7. The controller renders a Nunjucks view (pages) or returns JSON (API).
8. The error handler converts any thrown `AppError` into a safe response and logs details.

Public pages vs admin

- Public pages are server-rendered, cached in Redis as rendered HTML for anonymous visitors (5-15 minutes, cleared on publish), and cacheable at Cloudflare.
- The admin is server-rendered pages plus a JSON API under `/api/admin`. It is never cached, requires login and is marked `noindex`.
- Both use the same services, so rules are written once.

Folder structure

Every file below is one function, one component or one data set. Folder names show the module; file names show the action.

```
tourism-app/
├── package.json
├── .env.example
├── .eslintrc.cjs                  # max-lines 200 warn, 350 error
├── tailwind.config.js
├── docker-compose.yml             # app, worker, postgres, redis, nginx
├── Dockerfile
├── migrations/                    # one file per change
│   ├── 001_extensions_and_enums.js
│   ├── 002_users_roles.js
│   ├── 003_media.js
│   ├── 004_destinations.js
│   ├── 005_tours.js
│   ├── 006_itinerary_days.js
│   ├── 007_accommodations.js
```

```
│   ├── 008_bookings.js
│   ├── 009_enquiries.js
│   ├── 010_blog.js
│   ├── 011_pages_and_settings.js
│   ├── 012_reviews.js
│   ├── 013_translations.js
│   ├── 014_audit_log.js
│   └── 015_indexes.js
├── seeds/
│   ├── seedRoles.js
│   ├── seedAdminUser.js
│   └── seedDemoContent.js
├── public/                         # served by Nginx
│   ├── css/app.css                 # Tailwind build output
│   ├── js/                         # browser modules
│   │   ├── main.js
│   │   ├── nav.js
│   │   ├── gallery.js
│   │   ├── bookingForm.js
│   │   ├── searchBox.js
│   │   └── admin/
│   │       ├── dataTable.js
│   │       ├── richEditor.js
│   │       ├── mediaPicker.js
│   │       ├── itineraryBuilder.js
│   │       └── charts.js
│   └── img/                        # logo, icons, placeholders
├── src/
│   ├── server.js                   # starts HTTP server only
│   ├── worker.js                   # starts queue workers only
│   ├── app.js                      # builds Express app, mounts routes
│   ├── config/
│   │   ├── env.js                  # validates env vars with Zod
│   │   ├── database.js             # pg Pool
│   │   ├── redis.js                # ioredis client
│   │   ├── session.js
│   │   ├── i18n.js
│   │   ├── nunjucks.js
│   │   └── constants.js
│   ├── middleware/
│   │   ├── requestId.js
│   │   ├── securityHeaders.js
│   │   ├── rateLimit.js
│   │   ├── authRequired.js
│   │   ├── requireRole.js
│   │   ├── csrfProtect.js
│   │   ├── validate.js
│   │   ├── detectLanguage.js
│   │   ├── pageCache.js
│   │   ├── uploadImage.js
│   │   ├── notFound.js
│   │   └── errorHandler.js
│   ├── shared/
│   │   ├── errors/                 # AppError.js, NotFoundError.js,
ForbiddenError.js, ValidationError.js
```

```
│   │   ├── utils/                # slugify.js, paginate.js, formatPrice.js,
generateReference.js, sanitizeHtml.js
│   │   ├── db/                   # withTransaction.js, buildWhere.js,
buildPagination.js
│   │   ├── cache/                # cacheGet.js, cacheSet.js, cacheDelete.js,
cacheKey.js
│   │   └── logger.js
│   ├── modules/                  # one folder per feature (same shape in each)
│   │   ├── auth/
│   │   ├── users/
│   │   ├── tours/
│   │   ├── destinations/
│   │   ├── accommodations/
│   │   ├── bookings/
│   │   ├── enquiries/
│   │   ├── blog/
│   │   ├── pages/
│   │   ├── media/
│   │   ├── reviews/
│   │   ├── settings/
│   │   ├── search/
│   │   ├── dashboard/
│   │   └── audit/
│   ├── jobs/
│   │   ├── queues.js
│   │   ├── sendBookingEmail.js
│   │   ├── sendEnquiryEmail.js
│   │   ├── processImage.js
│   │   ├── purgeCdnCache.js
│   │   └── buildSitemap.js
│   ├── locales/
│   │   ├── en/ (common.json, tours.json, booking.json, admin.json)
│   │   └── sw/ (same files)
│   └── views/
│       ├── layouts/ (public.njk, admin.njk, auth.njk)
│       ├── partials/ (header.njk, footer.njk, seoHead.njk, pagination.njk,
flash.njk, tourCard.njk, destinationCard.njk, postCard.njk, languageSwitch.njk)
│       ├── pages/ (public pages, see Pages)
│       ├── admin/ (admin pages, see Pages)
│       └── errors/ (404.njk, 500.njk)
├── styles/
│   ├── input.css                 # @import tailwind + theme tokens
│   └── components/ (buttons.css, forms.css, cards.css, tables.css)
├── tests/
│   ├── unit/
│   ├── integration/
│   └── security/ (permissions.test.js, csrf.test.js, rateLimit.test.js)
├── scripts/ (backupDb.sh, restoreDb.sh, checkFileSize.js, createAdmin.js)
└── docs/ (api.md, runbook.md, adminGuide.md)
```

Shape of every module (example: `modules/tours/`)

```
modules/tours/
├── index.js                      # exports the module's public API only
├── tours.routes.public.js        # public routes
├── tours.routes.admin.js         # admin routes
```

```
├── controllers/
│   ├── listToursPage.js
│   ├── showTourPage.js
│   ├── adminListTours.js
│   ├── adminCreateTour.js
│   ├── adminUpdateTour.js
│   ├── adminPublishTour.js
│   └── adminDeleteTour.js
├── services/
│   ├── listTours.js
│   ├── getTourBySlug.js
│   ├── createTour.js
│   ├── updateTour.js
│   ├── publishTour.js
│   ├── archiveTour.js
│   └── invalidateTourCache.js
├── repositories/
│   ├── findTours.js
│   ├── findTourBySlug.js
│   ├── insertTour.js
│   ├── updateTourRow.js
│   ├── setTourStatus.js
│   └── findTourDays.js
└── validators/
    ├── tourListQuery.js
    ├── tourCreateBody.js
    └── tourUpdateBody.js
```

All 15 modules follow this shape. A module with more than about 12 files in a layer gets a sub-folder (for example `services/itinerary/`).

Security architecture

Security is layered: the edge (Cloudflare and Nginx), the application, the data stores and the server. No single control is trusted alone.

Threat	Control	Where
Password theft	argon2id hashing, minimum 12 characters, breached-password check, no password hints	`auth/services`
Brute-force login	5 attempts per 15 minutes per account and IP in Redis, then lock for 15 minutes, with a short delay on each failure	`middleware/rateLimit.js`
Session hijacking	Server-side sessions in Redis; cookie is `HttpOnly`, `Secure`, `SameSite=Lax`, `__Host-` prefix; session ID regenerated on login; 8-hour idle expiry; logout destroys	`config/session.js`

Threat	Control	Where
	the session	
CSRF	Synchroniser-token CSRF on every POST, PUT, PATCH, DELETE; token in a hidden field or `X-CSRF-Token` header	`middleware/csrfProtect.js`
SQL injection	Parameterised queries only (`$1`, `$2`), no string-built SQL, ORDER BY and column names from allowlists	`repositories/`, `shared/db/buildWhere.js`
XSS	Nunjucks auto-escaping on; rich text sanitised on save and on output (sanitize-html allowlist); strict Content-Security-Policy with nonces, no inline scripts	`utils/sanitizeHtml.js`, `securityHeaders.js`
Broken access control	Role checks on the server for every admin route; ownership checks in services; deny by default	`requireRole.js`
Mass assignment	Zod schemas strip unknown fields; services pick fields explicitly	`validators/`
File upload attacks	Limit size (8 MB) and count; check magic bytes, not just extension; re-encode with sharp; random file names; store outside the web root or in object storage	`uploadImage.js`, `processImage.js`
Spam and bots	Honeypot field, rate limit per IP, Cloudflare Turnstile on public forms	Public forms
Clickjacking and sniffing	`X-Frame-Options: DENY`, `frame-ancestors 'none'`, `nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`	`securityHeaders.js`
Transport	HTTPS only, HSTS with preload, TLS 1.2 or higher, Cloudflare Full (Strict)	Cloudflare, Nginx
Secrets leakage	`.env` never committed, validated at start, secrets rotated, no secrets	`config/env.js`

Threat	Control	Where
	in logs	
Information leakage	Generic error messages to users; stack traces only in logs; same message for unknown email or wrong password	`errorHandler.js`
Dependency risk	`npm audit` and Dependabot in CI, lockfile committed, pinned Node version	CI
Data loss	Daily PostgreSQL dump to external storage, 30-day retention, quarterly restore test; Redis persistence on (AOF)	`scripts/backupDb.sh`
Insider mistakes	Audit log of every admin write (who, what, before and after, IP); soft delete with restore	`modules/audit`
Server exposure	Only ports 80, 443 and SSH (key only, non-default port) open; PostgreSQL and Redis bound to the private Docker network with passwords; fail2ban; unattended security upgrades	VPS, `docker-compose.yml`
Privacy	Collect only needed booking data; privacy policy and cookie notice; data export and deletion on request	Pages, `bookings`

Admin hardening (recommended before launch): optional two-factor login (TOTP) for Manager and Super admin roles, an IP allowlist option for `/admin`, and an alert email on new admin logins.

Roles and permissions use a permission table (`resource:action`, for example `tours:publish`). Roles are sets of permissions, so new roles need no code change.

Database

PostgreSQL 16. Every table has `id` (UUID, `gen_random_uuid()`), `created_at` and `updated_at` (timestamptz). Content tables also have `status` (`draft`, `review`, `published`, `archived`), `deleted_at` for soft delete, and `created_by` / `updated_by` references to `users`. Money is stored as integer minor units with a currency code. Extensions: `pgcrypto`, `pg_trgm` (search), `unaccent`.

Table	Main columns	Constraints and notes

Table	Main columns	Constraints and notes
roles	name, description	Unique name
permissions	key (`tours:publish`)	Unique key
role_permissions	role_id, permission_id	Composite primary key
users	email, password_hash, full_name, role_id, is_active, failed_logins, locked_until, last_login_at, totp_secret	Unique lower(email); index on role_id
media	storage_key, url, mime_type, width, height, size_bytes, alt_text, folder, uploaded_by	Index on folder
destinations	name, slug, region, summary, description, latitude, longitude, best_season, hero_media_id, seo_title, seo_description, status	Unique slug; index on status
tours	title, slug, summary, description, duration_days, difficulty, category, price_from, currency, max_group_size, inclusions (jsonb), exclusions (jsonb), hero_media_id, is_featured, seo fields, status	Unique slug; indexes on status, category, is_featured; trigram index on title
tour_destinations	tour_id, destination_id	Composite primary key
tour_media	tour_id, media_id, sort_order	Composite primary key
itinerary_days	tour_id, day_number, title, description, overnight_place	Unique (tour_id, day_number)
price_tiers	tour_id, label, min_travellers, max_travellers, price_per_person, season	Check min <= max
accommodations	name, slug, type, destination_id, description, amenities (jsonb), price_range, hero_media_id, status	Unique slug
accommodation_media	accommodation_id, media_id, sort_order	Composite primary key
bookings	reference, tour_id, customer_name, customer_email, customer_phone, country, travel_date, adults, children, message, total_amount, currency, status,	Unique reference; indexes on status, travel_date, customer_email;

Table	Main columns	Constraints and notes
	internal_notes, assigned_to	check adults >= 1
booking_events	booking_id, type, note, actor_id	Timeline of status changes
enquiries	name, email, phone, subject, message, source_page, status, assigned_to, ip_hash	Indexes on status, created_at
post_categories	name, slug	Unique slug
posts	title, slug, excerpt, body_html, category_id, author_id, hero_media_id, published_at, seo fields, status	Unique slug; index on (status, published_at)
post_tags / tags	name, slug; post_id, tag_id	
pages	slug, title, sections (jsonb), seo fields, status	Unique slug (home, about, contact, terms, privacy)
reviews	tour_id, author_name, country, rating, body, is_approved	Check rating 1-5
settings	key, value (jsonb)	Unique key (logo, colours, contact, social, email templates)
translations	entity, entity_id, language, field, text	Unique (entity, entity_id, language, field)
newsletter_subscribers	email, confirmed_at, unsubscribed_at	Unique lower(email)
content_versions	entity, entity_id, snapshot (jsonb), version_number, created_by	Enables revert
audit_log	user_id, action, entity, entity_id, before (jsonb), after (jsonb), ip, user_agent	Append-only; index on (entity, entity_id) and created_at

Search: a generated `tsvector` column plus a GIN index on tours, destinations and posts, with `pg_trgm` for typo-tolerant title matching.

Migrations: 15 files, listed in the folder structure; each has `up` and `down`, runs in a transaction and stays under 200 lines (a large table is its own file).

Connection settings: pool of 10 connections per process, 30-second statement timeout, `ssl` required if the database is ever moved off the private network.

Backups and restore: `pg_dump` nightly, encrypted, uploaded off-server; `restoreDb.sh` restores into a scratch database for the quarterly test.

Redis

Redis holds only data that can be rebuilt, except queue jobs and sessions. All keys use the prefix `tour:` so one Redis instance can safely serve other apps, and every key has a time to live (TTL).

Purpose	Key pattern	TTL	Cleared when
Sessions	`tour:sess:{sessionId}`	8 h idle	Logout or expiry
Login throttling	`tour:rl:login:{ip}:{email}`	15 min	Expiry
API and form rate limits	`tour:rl:{route}:{ip}`	1-60 min	Expiry
Rendered public HTML	`tour:page:{lang}:{path}:{queryHash}`	10 min	Publish or edit of related content
Lists and lookups	`tour:list:{resource}:{lang}:{queryHash}`	5 min	Write to that resource
Single items	`tour:item:{resource}:{slug}:{lang}`	15 min	Write to that item
Settings	`tour:settings`	1 h	Settings saved
Dashboard stats	`tour:stats:{range}`	2 min	Expiry
Search suggestions	`tour:suggest:{lang}:{term}`	10 min	Expiry
Queues (BullMQ)	`bull:{queueName}:*`	Managed	Job complete

Cache rules

- Only anonymous GET requests are cached. Any request with a session cookie skips the page cache.
- A service that changes content calls `invalidate{Resource}Cache`, which deletes the item, list and page keys by prefix (using `SCAN`, never `KEYS`) and queues `purgeCdnCache` for the changed URLs.
- If Redis is down, the app serves from PostgreSQL directly and logs a warning; login throttling fails closed (blocks) for the admin area.

Queues: `email` (booking and enquiry notices, 5 retries with backoff), `images` (resize and convert), `cdn` (purge Cloudflare), `sitemap` (rebuild). The worker runs as its own process so slow jobs never block web requests.

Persistence: AOF on with `everysec`, password required, bound to the private network, `maxmemory` set with `allkeys-lru` for cache keys.

API reference

Pages are server-rendered. JSON endpoints serve forms, the admin interface and any future mobile app. Base path `/api/v1`. JSON responses use one shape.

```
{ "ok": true, "data": { }, "meta": { "page": 1, "pageSize": 20, "total": 134 } }
{ "ok": false, "error": { "code": "VALIDATION_ERROR", "message": "Invalid input",
"fields": { "email": "Enter a valid email" } } }
```

Conventions: lists accept `page`, `pageSize` (max 100), `sort` (allowlisted fields, `-` prefix for descending), `q` (search) and resource filters. Status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500. Admin writes need the session cookie and `X-CSRF-Token`. Language comes from the `/{lang}/` URL prefix or the `Accept-Language` header.

Authentication

Method	Path	Access	Purpose
POST	`/api/v1/auth/login`	Public, rate limited	Start session (email, password, optional TOTP code)
POST	`/api/v1/auth/logout`	Signed in	Destroy session
GET	`/api/v1/auth/me`	Signed in	Current user and permissions
POST	`/api/v1/auth/forgot-password`	Public, rate limited	Send reset link (same response whether email exists)
POST	`/api/v1/auth/reset-password`	Public, token	Set new password
POST	`/api/v1/auth/change-password`	Signed in	Change own password
POST	`/api/v1/auth/accept-invite`	Public, token	Set password for an invited user
GET	`/api/v1/auth/csrf`	Public	Fetch a CSRF token

Public API

Method	Path	Purpose
GET	`/api/v1/tours`	List published tours (filters: destination, category, duration, price range, difficulty, featured)
GET	`/api/v1/tours/:slug`	Tour detail with itinerary, prices, media and reviews
GET	`/api/v1/destinations`	List published destinations
GET	`/api/v1/destinations/:slug`	Destination detail and related tours
GET	`/api/v1/accommodations`	List accommodations (filters: destination, type)

Method	Path	Purpose
GET	`/api/v1/accommodations/:slug`	Accommodation detail
GET	`/api/v1/posts`	List published articles (filters: category, tag)
GET	`/api/v1/posts/:slug`	Article detail
GET	`/api/v1/gallery`	Gallery items grouped by destination
GET	`/api/v1/search?q=`	Search across tours, destinations and posts
GET	`/api/v1/search/suggest?q=`	Autocomplete (max 8 results)
POST	`/api/v1/bookings`	Create a booking request (Turnstile and rate limited)
POST	`/api/v1/enquiries`	Send contact enquiry (Turnstile and rate limited)
POST	`/api/v1/newsletter`	Subscribe
POST	`/api/v1/reviews`	Submit a review for moderation
GET	`/api/v1/settings/public`	Site name, contacts, social links, currency

Admin API (all under `/api/v1/admin`, signed in; permission shown)

Method	Path	Permission	Purpose
GET	`/dashboard/summary`	`dashboard:read`	KPIs: bookings, enquiries, revenue estimate, top tours
GET	`/dashboard/chart?range=`	`dashboard:read`	Time series for charts
GET, POST	`/tours`	`tours:read`, `tours:create`	List, create
GET, PATCH, DELETE	`/tours/:id`	`tours:read`, `tours:update`, `tours:delete`	Read, update, soft delete
POST	`/tours/:id/publish`, `/unpublish`, `/duplicate`	`tours:publish`, `tours:create`	Workflow actions
PUT	`/tours/:id/itinerary`	`tours:update`	Replace itinerary days
PUT	`/tours/:id/prices`	`tours:update`	Replace price tiers
GET	`/tours/:id/versions`, POST `/tours/:id/revert/:versionId`	`tours:update`	Version history, revert
GET, POST	`/destinations`, GET, PATCH, DELETE `/destinations/:id`, POST	`destinations:*`	Same pattern as tours

Method	Path	Permission	Purpose
	`/destinations/:id/publish`		
GET, POST	`/accommodations`, GET, PATCH, DELETE `/accommodations/:id`, POST `/accommodations/:id/publish`	`accommodations:*`	Same pattern
GET, POST	`/posts`, GET, PATCH, DELETE `/posts/:id`, POST `/posts/:id/publish`, `/schedule`	`posts:*`	Articles, scheduling
GET, POST, PATCH, DELETE	`/post-categories`, `/tags`	`posts:update`	Taxonomy
GET, PATCH	`/pages`, `/pages/:slug`	`pages:read`, `pages:update`	Edit static pages and home sections
GET	`/bookings`	`bookings:read`	List with filters and CSV export (`?format=csv`)
GET, PATCH	`/bookings/:id`	`bookings:read`, `bookings:update`	Detail, update notes and assignment
POST	`/bookings/:id/status`	`bookings:update`	Change status (writes a booking event, emails customer)
GET	`/enquiries`, GET, PATCH `/enquiries/:id`, POST `/enquiries/:id/reply`	`enquiries:*`	Inbox, assign, reply
GET, POST	`/media`	`media:read`, `media:create`	List, upload (multipart)
PATCH, DELETE	`/media/:id`	`media:update`, `media:delete`	Edit alt text and folder, delete if unused
GET	`/reviews`, POST `/reviews/:id/approve`, `/hide`	`reviews:*`	Moderation
GET, PATCH	`/settings`	`settings:read`, `settings:update`	Site settings
GET, POST	`/users`, PATCH, DELETE `/users/:id`, POST `/users/:id/invite`, `/reset-2fa`	`users:*`	Staff management

Method	Path	Permission	Purpose
GET, POST	`/roles`, PATCH `/roles/:id`	`roles:*`	Roles and permissions
GET	`/audit-log`	`audit:read`	Filter by user, entity, date
POST	`/cache/purge`	`settings:update`	Clear page cache and CDN

Webhooks and health: `GET /healthz` (liveness) and `GET /readyz` (checks PostgreSQL and Redis) are public and return no details. A Cloudflare Turnstile verification is required on `POST /bookings`, `/enquiries`, `/newsletter` and `/reviews`.

Pages

Public URLs are prefixed with the language (`/en/...`, `/sw/...`); `/` redirects to the visitor's language. Every public page extends `layouts/public.njk`, includes `partials/seoHead.njk` (title, description, canonical, hreflang, Open Graph, JSON-LD) and is cached as rendered HTML.

Public pages

Route	Template	Data loaded
`/{lang}`	`pages/home.njk`	Page sections, featured tours, top destinations, latest posts, approved reviews
`/{lang}/destinations`	`pages/destinations/list.njk`	Published destinations, filters
`/{lang}/destinations/:slug`	`pages/destinations/show.njk`	Destination, media, related tours, nearby accommodation
`/{lang}/tours`	`pages/tours/list.njk`	Paginated tours, filters, sort
`/{lang}/tours/:slug`	`pages/tours/show.njk`	Tour, itinerary, price tiers, media, reviews, related tours; booking form
`/{lang}/book/:slug`	`pages/booking/form.njk`	Tour summary, form with Turnstile
`/{lang}/booking/confirmation/:reference`	`pages/booking/confirmation.njk`	Booking summary (reference and email check)
`/{lang}/accommodations`	`pages/accommodations/list.njk`	Properties, filters
`/{lang}/accommodations/:slug`	`pages/accommodations/show.njk`	Property, media, amenities, enquiry button
`/{lang}/gallery`	`pages/gallery.njk`	Albums by destination

Route	Template	Data loaded
`/{lang}/blog`	`pages/blog/list.njk`	Posts, categories, tags, pagination
`/{lang}/blog/:slug`	`pages/blog/show.njk`	Post, related posts
`/{lang}/blog/category/:slug`, `/tag/:slug`	`pages/blog/list.njk`	Filtered posts
`/{lang}/about`	`pages/static.njk`	Page record `about`
`/{lang}/contact`	`pages/contact.njk`	Settings, enquiry form with Turnstile
`/{lang}/search?q=`	`pages/search.njk`	Combined search results
`/{lang}/terms`, `/privacy`	`pages/static.njk`	Page records
`/sitemap.xml`, `/robots.txt`	Generated	Published URLs in all languages
404, 500	`errors/404.njk`, `errors/500.njk`	None

Auth pages (`layouts/auth.njk`, `noindex`)

Route	Template
`/admin/login`	`admin/auth/login.njk`
`/admin/forgot-password`	`admin/auth/forgot.njk`
`/admin/reset-password/:token`	`admin/auth/reset.njk`
`/admin/accept-invite/:token`	`admin/auth/invite.njk`

Admin pages (`layouts/admin.njk`, sidebar by permission, never cached)

Route	Template	Main components
`/admin`	`admin/dashboard.njk`	KPI cards, charts, recent bookings and enquiries, activity feed
`/admin/tours`	`admin/tours/list.njk`	Data table, filters, bulk actions
`/admin/tours/new`, `/admin/tours/:id/edit`	`admin/tours/form.njk`	Tabs: details, itinerary builder, prices, media, SEO, translations, versions
`/admin/destinations` (+ new, edit)	`admin/destinations/*.njk`	Table, form with map pin picker
`/admin/accommodations` (+	`admin/accommodations/*.njk`	Table, form

Route	Template	Main components
new, edit)		
`/admin/posts` (+ new, edit)	`admin/posts/*.njk`	Table, rich editor, schedule, categories, tags
`/admin/pages`, `/admin/pages/:slug/edit`	`admin/pages/*.njk`	Section editor for home and static pages
`/admin/bookings`, `/admin/bookings/:id`	`admin/bookings/*.njk`	Table, status timeline, notes, email customer, export
`/admin/enquiries`, `/admin/enquiries/:id`	`admin/enquiries/*.njk`	Inbox, assign, reply
`/admin/media`	`admin/media/library.njk`	Grid, upload, folders, alt text
`/admin/reviews`	`admin/reviews/list.njk`	Approve, hide
`/admin/users`, `/admin/roles`	`admin/users/*.njk`	Staff, invitations, roles, permissions
`/admin/settings`	`admin/settings/*.njk`	Tabs: general, branding, contact, email, languages, SEO
`/admin/audit-log`	`admin/audit/list.njk`	Filterable log
`/admin/profile`	`admin/profile.njk`	Name, password, two-factor setup

Shared Nunjucks partials (`tourCard`, `dataTable`, `formField`, `statusBadge`, `pagination`, `confirmModal`) keep each template under 200 lines.

Database integration

SQL appears only in repository files. Services add rules, caching and transactions. Below is one complete path (list published tours); each block is its own file and stays well under 200 lines.

`src/config/database.js`

```
import pg from 'pg';
import { env } from './env.js';

export const pool = new pg.Pool({
  connectionString: env.DATABASE_URL,
  max: 10,
  statement_timeout: 30000,
});
```

`src/shared/db/withTransaction.js`

```
import { pool } from '../../config/database.js';
```

```
export async function withTransaction(work) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const result = await work(client);
    await client.query('COMMIT');
    return result;
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release();
  }
}
```

src/modules/tours/repositories/findTours.js

```
import { pool } from '../../../config/database.js';

const SORTS = { newest: 't.created_at DESC', price: 't.price_from ASC' };

export async function findTours({ category, limit, offset, sort = 'newest' }, db =
pool) {
  const { rows } = await db.query(
    `SELECT t.id, t.slug, t.title, t.summary, t.duration_days, t.price_from,
            m.url AS hero_url, COUNT(*) OVER() AS total
       FROM tours t LEFT JOIN media m ON m.id = t.hero_media_id
      WHERE t.status = 'published' AND t.deleted_at IS NULL
        AND ($1::text IS NULL OR t.category = $1)
      ORDER BY ${SORTS[sort] ?? SORTS.newest}
      LIMIT $2 OFFSET $3`,
    [category ?? null, limit, offset],
  );
  return rows;
}
```

src/modules/tours/services/listTours.js

```
import { findTours } from '../repositories/findTours.js';
import { cacheGet } from '../../../shared/cache/cacheGet.js';
import { cacheSet } from '../../../shared/cache/cacheSet.js';
import { cacheKey } from '../../../shared/cache/cacheKey.js';
import { paginate } from '../../../shared/utils/paginate.js';

export async function listTours(query, lang) {
  const key = cacheKey('list', 'tours', lang, query);
  const cached = await cacheGet(key);
  if (cached) return cached;

  const page = paginate(query);
  const rows = await findTours({ ...query, ...page });
  const result = { items: rows, total: Number(rows[0]?.total ?? 0), ...page };

  await cacheSet(key, result, 300);
  return result;
}
```

src/modules/tours/validators/tourListQuery.js

```
import { z } from 'zod';

export const tourListQuery = z.object({
  category: z.string().max(40).optional(),
  sort: z.enum(['newest', 'price']).optional(),
  page: z.coerce.number().int().min(1).default(1),
  pageSize: z.coerce.number().int().min(1).max(100).default(12),
});
```

src/modules/tours/controllers/listToursPage.js

```
import { listTours } from '../services/listTours.js';

export async function listToursPage(req, res) {
  const data = await listTours(req.validated.query, req.lang);
  res.render('pages/tours/list.njk', { data, query: req.validated.query });
}
```

src/modules/tours/tours.routes.public.js

```
import { Router } from 'express';
import { validate } from '../../middleware/validate.js';
import { pageCache } from '../../middleware/pageCache.js';
import { tourListQuery } from './validators/tourListQuery.js';
import { listToursPage } from './controllers/listToursPage.js';

export const toursPublicRoutes = Router();
toursPublicRoutes.get('/tours', pageCache(600), validate({ query: tourListQuery }),
listToursPage);
```

Write path (create a booking), the pattern for all writes

1. The route applies, in order: `rateLimit`, `verifyTurnstile`, `validate({ body })`.
2. The controller `createBookingApi` calls `services/createBooking.js`.
3. The service opens `withTransaction`, checks the tour is published, inserts the booking with `insertBooking(client, ...)`, inserts a `booking_events` row, and commits.
4. After commit it queues `sendBookingEmail` for the customer and staff, and returns the reference.
5. The controller answers `201` with the reference, or redirects to the confirmation page.
6. On any thrown error the transaction rolls back and `errorHandler` returns a safe message.

Rules: repositories accept an optional `db` argument (pool or transaction client) so services can combine them in one transaction; services never build SQL; controllers never import repositories; migrations are the only way the schema changes.

Environment, deployment and testing

Environment variables (validated at start by `config/env.js`; the app refuses to boot if one is missing)

Variable	Purpose
`NODE_ENV`, `PORT`, `APP_URL`	Runtime and public URL
`DATABASE_URL`	PostgreSQL connection string

Variable	Purpose
`REDIS_URL`	Redis connection string with password
`SESSION_SECRET`	64+ random characters, rotated on a schedule
`CSRF_SECRET`	Separate random secret
`SMTP_HOST`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	Outgoing email
`STORAGE_DRIVER`, `STORAGE_BUCKET`, `STORAGE_KEY`, `STORAGE_SECRET`	Image storage (local or S3-compatible)
`TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET`	Bot protection
`CLOUDFLARE_ZONE_ID`, `CLOUDFLARE_API_TOKEN`	Cache purge
`DEFAULT_LANG`, `SUPPORTED_LANGS`	`en`, `en,sw`

Docker Compose services: `web` (Node, port 3000 internal), `worker` (same image, runs `worker.js`), `postgres` (volume, private network), `redis` (volume, password), `nginx` (ports 80 and 443). Only Nginx is exposed. Coolify or Dokploy can manage the same services.

Build and release

1. `npm ci`, then `npm run lint` (includes the file-size check), `npm test`.
2. `npm run build:css` produces the minified Tailwind file with a content hash in its name.
3. Build the Docker image, run `npm run migrate up` once, then start `web` and `worker`.
4. A health check on `/readyz` gates traffic; the previous container stays up until the new one is healthy.
5. Rollback: redeploy the previous image tag; migrations always have a `down`.

NPM scripts: `dev` (nodemon and Tailwind watch), `build:css`, `start`, `worker`, `migrate`, `seed`, `lint`, `test`, `check:size` (fails if any file exceeds 350 lines, warns above 200).

Testing

Level	Tool	What is covered
Unit	Vitest	Utilities, validators, service rules with mocked repositories
Integration	Vitest and Supertest on a test database and Redis	Every API route, transactions, cache invalidation
Security	Supertest	Every admin route returns 401 or 403 without the right role; CSRF missing returns 403; rate limits trigger; uploaded fake images are rejected
End to end	Playwright	Booking request, admin login, create and publish a tour, status change

Level	Tool	What is covered
Performance	Lighthouse CI	Mobile score of 85 or higher on home, tour and blog pages

Monitoring and operations: pino logs shipped to a file with rotation, Sentry for errors, an uptime monitor on `/readyz`, alerts for failed jobs and disk above 80 percent, and a runbook (`docs/runbook.md`) for restore, rollback and key rotation.

Build order (maps to the 14-day schedule in the blueprint): config, database, migrations and auth (days 1-3); shared utilities, admin shell and the tours, destinations and media modules (days 4-6); public layouts and pages (days 7-8); bookings, enquiries and email queue (day 9); dashboard, audit log and settings (day 10); i18n and SEO (day 11); content, polish, tests and security review (days 12-13); deployment and handover (day 14).